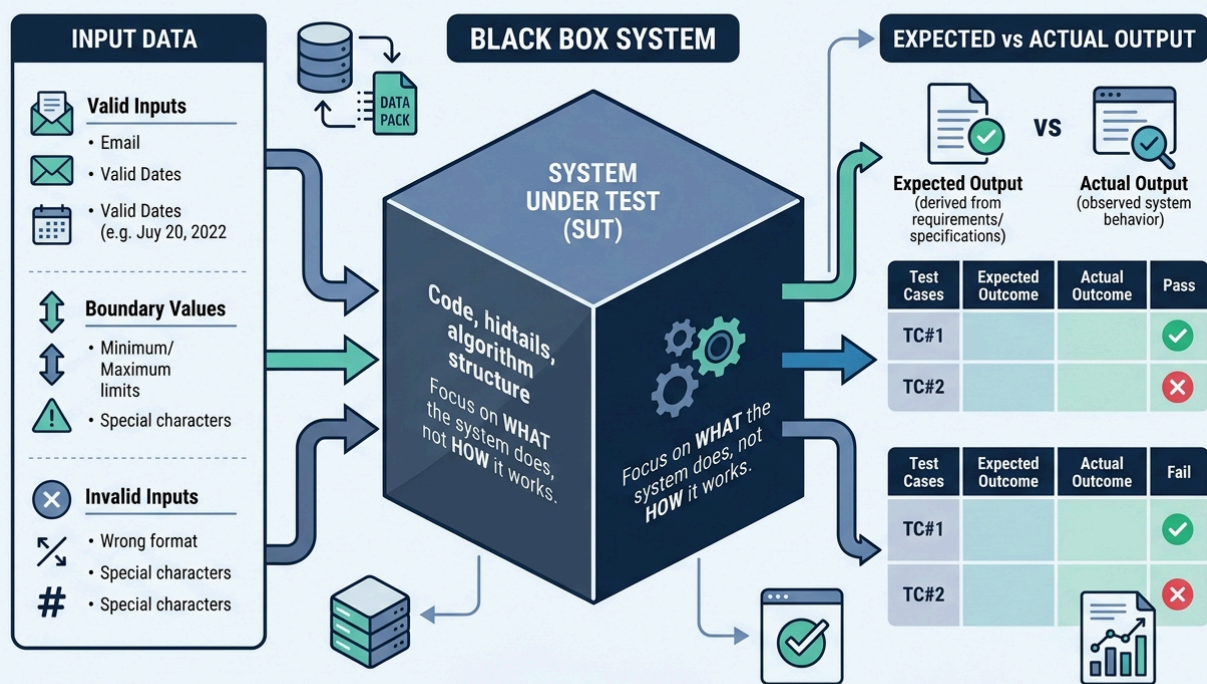


Ch05 黑箱測試

黑箱測試是在不知道程式內部結構的情況下進行測試。

Black-box testing A method of software testing that examines the functionality of an application without peering into its internal structures or workings.

UNDERSTANDING BLACK-BOX TESTING: A FUNCTIONAL APPROACH



圖形解說：黑箱測試 (Black-Box Testing) 概念架構

- **【左側】測試輸入 (Input Data)**：包含有效輸入 (Valid Inputs)、邊界值 (Boundary Values) 與無效/格式錯誤輸入 (Invalid Inputs)。
- **【中央】受測系統 (System Under Test, SUT)**：系統內部程式碼、資料結構與演算法細節對測試人員隱藏不透明（黑盒子），聚焦於系統「做什麼 (What)」，而非「如何做 (How)」。
- **【右側】輸出驗證 (Expected vs. Actual Output)**：比對受測系統實際產出的行為 (Actual Output) 是否完全符合需求規格所定義的預期結果 (Expected Output)。

💡 軟體測試之父：Glenford J. Myers

1979 年，美國電腦科學家 **Glenford J. Myers** 出版了軟體工程史上的奠基經典巨著《**The Art of Software Testing**》（軟體測試的藝術）。Myers 提出了一句徹底顛覆軟體業界的革命性名言：「**測試是為了發現錯誤而執行程式的過程，而不是為了證明程式沒有錯誤。**」
(*Testing is the process of executing a program with the intent of finding errors.*) 他破除了「測試全部通過 = 系統毫無瑕疵」的盲目樂觀，率先將黑箱測試三大核心技術——**等價類分割 (Equivalence Partitioning)**、**邊界值分析 (Boundary Value Analysis)** 與 **錯誤猜測 (Error Guessing)** 系統化。書中所提出的「經典三角形分類問題 (The Triangle Problem)」更成為五十年來全球資訊界檢驗黑箱測試案例完整性的教科書級標竿。

📌 本章目錄與重點導讀 (Table of Contents & Highlights)

黑箱測試 (Black-Box Testing) 是從使用者與規格角度出發的行為驗證技術。本章完整涵蓋軟體工程最經典的輸入劃分、組合優化、邏輯決策與現代屬性測試：

Ch05 知識架構全景：

【自動化基石】5.1 JUnit 5 實戰 & 斷言機制

【輸入域劃分】5.2 邊界值分析 (BVA: $4n+1, 6n+1$) & 5.3 等價類分割 (EP: 弱/強涵蓋)

【組合爆炸解法】5.4 全成對測試 (Pairwise / All-Pairs) & 5.5 案例優化

【複雜業務邏輯】5.6 決策表測試 (Decision Table) & 規則化簡

【事件驅動架構】5.7 狀態轉換測試 (State Transition / 1-Switch)

【現代前沿測試】5.8 屬性基礎測試 (Property-Based Testing with jqwik)

章節單元	核心學習重點 (Key Takeaways)
5.1 Lab: JUnit	掌握 JUnit 5 單元測試基礎與斷言機制 (<code>assertEquals</code> 內容相等 vs. <code>assertSame</code> 記憶體同一性)。
5.2 邊界測試 (BVA)	掌握「錯誤隱藏在角落」的核心理念；精熟 獨立型 vs. 非獨立型 、 標準邊界 (4n+1) vs. 強固邊界 (6n+1 / 7n+1) 測試案例設計。
5.3 等價分割測試 (EP)	掌握有效等價類與無效等價類劃分；精熟 弱一般/弱強固 vs. 強一般/強強固 等價測試之選取準則。
5.4 全成對組合測試 (Pairwise)	解決多參數輸入的「組合爆炸 (Combinatorial Explosion)」難題；掌握 All-Pairs / 正交表 原理與工具 (PICT / ACTS)。
5.5 測試案例的優化	學習在覆蓋率不降低的前提下消除冗餘案例，達到測試集精實化。
5.6 決策表測試 (Decision Table)	針對複雜業務邏輯，掌握 條件樁 (Condition Stub) 、 動作樁 (Action Stub) 、規則展開與布林邏輯規則化簡。
5.7 狀態測試 (State Testing)	針對物件生命週期與狀態機 (FSM)，設計狀態轉換矩陣與 0-Switch / 1-Switch 覆蓋路徑。
5.8 屬性基礎測試 (PBT)	掌握 SQA 2.0 現代前沿技術：使用 <code>jqwik</code> 定義核心數學不變量，自動生成海量隨機測資並執行 測資收縮 (Shrinking) 。
5.9 實戰練習題	涵蓋游泳池計費、青年成家貸款、電子表狀態機與三角形多維案例實戰。

5.1 Lab: JUnit

[JUnit on GitHub](#)

概念核對問答 (CCQ 1)

- **是非題**：在 JUnit 中，`assertSame(a, b)` 斷言的作用與 `assertEquals(a, b)` 完全相同，都是在驗證兩個物件的內容值是否相等（即比對 `a.equals(b)`）。

► [點擊查看【概念核對問答】答案與解析](#)

[課堂互動](#)

5.2 邊界測試

錯誤都隱藏在角落！！

獨立型邊界測試

獨立型邊界測試（Independent Boundary Value Testing）是邊界測試中最基本的測試。這裡所謂的獨立，是系統內的變數彼此獨立，不會聯合起來影響系統的功能或行為。所以如果程式在一個變數邊界的地方寫錯了，錯誤就會彰顯出來，我們不需要特別去測試兩個變數同時在邊界的狀況。

舉例來說，假設一個系統輸入學生的分數與身高，若分數 ≥ 90 則成績為 A，身高若高於 180 則建議參加籃球隊：

```
if (s > 90) //應為 >=
    grade = "a";
if (tall > 180)
    exercise = "baseball";
```

分數與身高兩個變數是獨立無關的，我們無需測試 $s=90$ 同時 $tall=180$ 的邊界測試。像這樣的情況我們可以說「單一錯誤假設」成立：

單一錯誤假設 一個變數的錯誤就足以造成系統的錯誤，不需要同時兩個（或以上）的失誤。

一個系統內會有很多的因子（或變數），邊界測試針對數字性的因子去檢驗其邊界值。假設變數的值在 min, max 範圍之內，則針對因子的邊界值進行測試: min, min+, max, max-, norm。其中 norm 表示 normal 就是在 min 與 max 之間取一個一般的代表值。假設有 n 個變數，每個變數有 min, min+, max, max- 等四個值，再加上一個非邊界的代表值，共是 $4n + 1$ 個測試資料。

針對每個數字性的因子的邊界：min, min+, max-, max 進行測試。

以下範例說明測試 max 的必要性：

程式碼寫法	缺陷	測試結果
if (input <= 10)	無缺陷	max=10 通過
if (input < 10)	缺陷：max=10 被錯誤排除。	max=10 測試會失敗。

[!NOTE] 考慮一個三角形判斷的程式，輸入三邊長 a, b, c，其邊長介於 1 與 200 之間。輸出為正三角形、等腰三角形、不等邊三角形、非三角形等四種可能。請依據邊界測試設計你的測試案例。

由於有三個變數，我們先固定兩個變數，讓其值為 normal (200)，然後變化另一個變數的值 (min, min+, max-, max)。最後再補上一個 normal 的值就完成了。如表所示。

case#	a	b	c	輸出
1	100	100(norm)	1(min)	等腰
2	100	100(norm)	2(min+)	等腰
3	100	100(norm)	199(max-)	等腰
4	100	100(norm)	200(max)	!
5	100	1	100	等腰
6	100	2	100	等腰
7	100	199	100	等腰
8	100	200	100	!
9	1	100	100	等腰
10	2	100	100	等腰
11	199	100	100	等腰
12	200	100	100	!
13	100	100	100	正三角形

這樣一組測試資料是不太夠的，系統規格中有提到要判斷是否為不等邊三角形，但因為我們著重在輸入變數的邊界測試，忽略掉變數的多樣性，只能測試出正三角形或等腰三角形；

解決的方法有兩種：

- 隨機的 normal 值。norm 的值不要固定，而是在 min+ 與 max- 之間選一個動態的產出，以提升數值的多樣性。
- 以輸出為基準。不只是以輸入為基礎來設計邊界資料，也把「輸出」考慮在內，規格中的輸出有正三角形、等腰三角形、不等邊三角形、非三角形等四種，設計設計案例使之必定這樣的輸出。

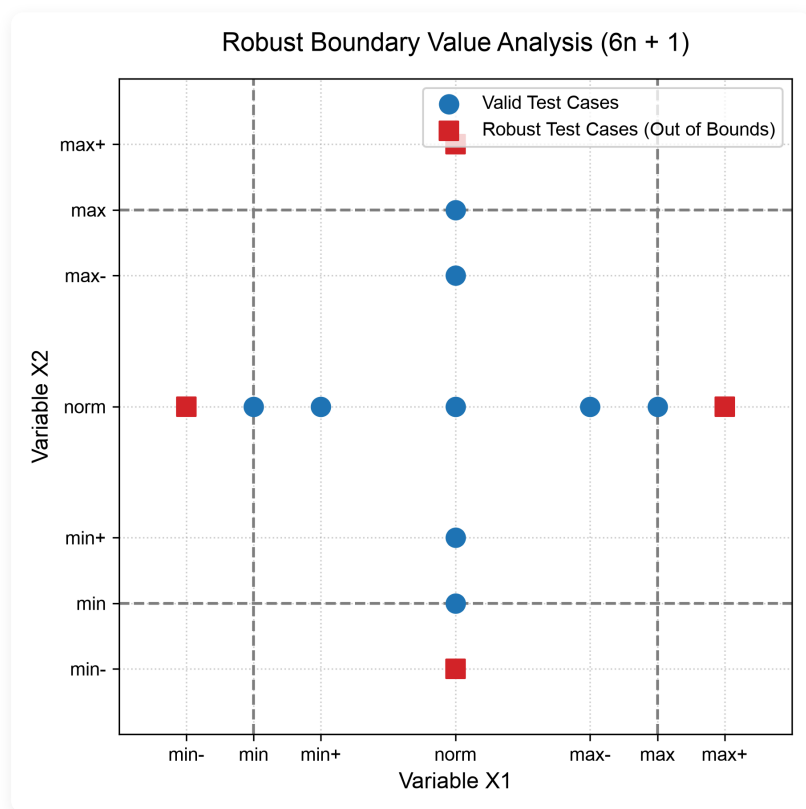
獨立型強固邊界測試

強固邊界測試 (Robust boundary value testing) 擴充一般邊界測試，多考慮例外情況，亦即 min-, max+ 等兩個狀況，所以共測試 $6n + 1$ 個狀況。透過檢視系統對例外的處理狀況來檢視系統的強固性。如圖 所示。

同樣以三角形的例子來解說，表中我們新增了兩個測試，分別是 min- 與 max+ 的測試案例。其中！表示例外的輸出。

case#	a	b	c	輸出
1	100	100(norm)	0(min-)	！
2	100	100(norm)	1(min)	等腰
3	100	100(norm)	2(min+)	等腰
4	100	100(norm)	199(max-)	等腰
5	100	100(norm)	200(max)	！
6	100	100(norm)	201(max+)	！
...	...	...	...	...
19	...	...	...	...

強固邊界測試



上述的三角形例子是一個比較單純的例子，每一個變數的合法範圍只有一個，有時候不同的範圍會有不同的輸出，所以就具備多個邊界值。

SwimmingPool

[!NOTE] 🏀 以下是逢甲游泳池收費系統的規則：

- (1) 一般票價 200
- (2) 星期六日250元,除會員以外不打折
- (3) 12歲以下、60歲（含）以上打八折，限定 3-75 歲可入內游泳
- (4) 七點以前八折
- (5) 團體打七折
- (6) 會員打五折
- (7) 各打折不得重疊使用，以顧客最有利方案定價

採用「獨立強固邊界測試」設計測試案例

- 有哪些變數？
- 哪些變數非列舉，有明顯的邊界？其 min, min-, max, max+ 為何？
- 請透過 excel，採用強固邊界測試進行測試案例規劃
- 請透過 JUnit 進程式碼測試

非獨立型邊界測試

有一些系統並不符合「單一錯誤假設 (single fault assumption)」。例如有一個評分系統具備「考試分數 <60 且 作業分數 < 60 則成績為 D，否則為 C」這樣的規則，邊界值為 60, 因此設計出 (60, 80), (80, 60) 的測試案例，兩個的結果都為應為 C。但程式碼犯了錯誤：

```
grade = "C";  
if (exam <= 60 && hw <= 60 ) //錯了  
    grade = "D"
```

我們把 < 都打成了 <=，但執行出來的結果一樣是 C，找不出錯誤。如果我們的測試案例都考慮邊界值 (60, 60), 這個錯誤就可以被找出來了。我們之所以用 (60, 80) 而非用 (60, 60) 來測試是因為我們相信一個 60 的邊界值就足以呈現錯誤（單一錯誤假設），但顯然在某些問題上並不是的。exam 和 hw 並非獨立，他們息息相關，在邏輯的判斷上他們同時出現。

非獨立型一般邊界測試又稱為 *最差情況邊界測試* (worst case boundary value testing) (真倒楣，兩個錯誤同時在邊界時產生)。這種情況，我們就必須用「乘積」的方式來設計測試案例。每一個變數有 min, min+, norm, max-, max 等五種狀況，所以共需要 5^n 個測試案例。這是一個不小的數目，如果有五個變數，則需要 $5^5 = 3,125$ 個測試案例。表 X 表示三角形程式在非獨立一般邊界測試時的狀況，因為是做乘積，總共會有 $5*5*5=125$ 個測試案例。

當變數很多時，其所需要的測試資料數量是我們無法負擔的，所以有時候也進行部分的乘積，而不是全面的，例如我們猜測其中三個高度相關，僅對這三個變數進行乘積，其餘的還是用獨立式的作法。

FIG: 非獨立型一般邊界測試 (最差情況邊界測試)

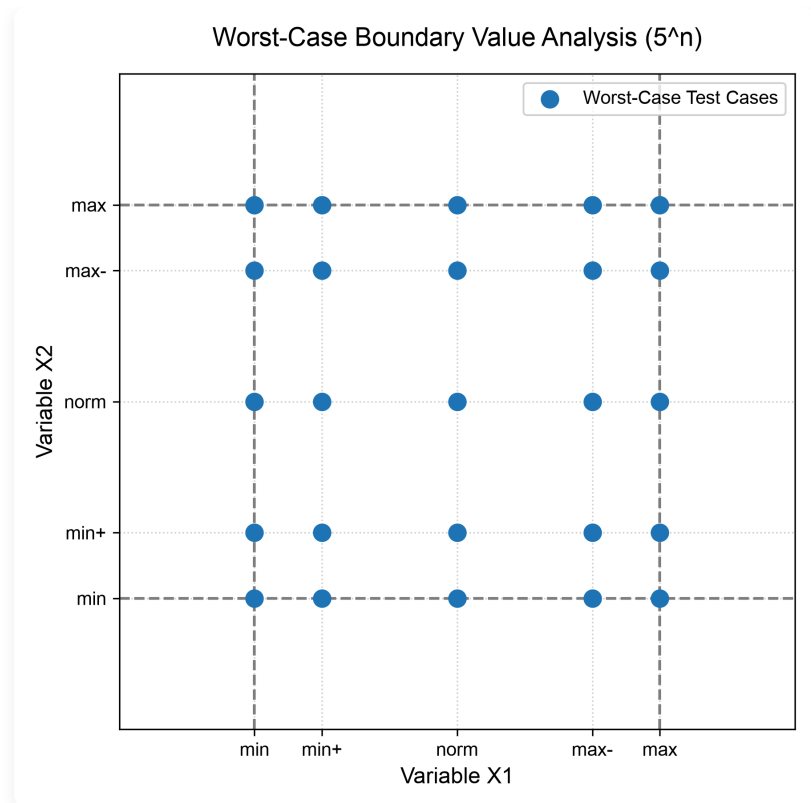


FIG: 三角形程式：非獨立型一般邊界測試

case#	a	b	c	輸出
1	100	100(norm)	1(min)	等腰
2	100	100(norm)	2(min+)	等腰
3	100	100(norm)	199(max-)	等腰
4	100	100(norm)	200(max)	!
5	100	100(norm)	100(norm)	正三角
6	100	1 (min)	1(min)	!
...	...	...	...	...
125	...	...	...	...

非獨立型強固邊界測試

同上，這次我們考慮強固的方案，亦即例外的狀況 max+, min- 考慮進去，所以共需要 7^n 個測試案例，因為每個變數會有以下 7 個資料：

- max+
- max

- max-
- normal
- min+
- min
- min-

當然，這樣的情況測試案例就會更多，以五個變數而言，供需要 $7^5 = 16,807$ 個測試案例。

總結

在設計邊界測試時應該考慮以下因素：

- 是否考慮強固性：是否考慮每一個變數的例外。
- 輸入、輸出、或是混和：以輸入的變數作為分割的基準，還是以輸出的變數？還是兩者都考慮？
- 獨立或非獨立：如果是獨立變數，我們的測試案例只要能夠「涵蓋」該邊界即可。若是非獨立，測試案例必須包含所有邊界的「乘積」。

不同的狀況所需建立的測試案例數量有所不同：

- 獨立型邊界測試: $4n + 1$
- 獨立型強固邊界測試: $6n + 1$
- 非獨立型邊界測試: 5^n
- 非獨立型強固邊界測試: 7^n

邊界測試時並不知曉變數值的意義，而是單純的範圍內的邊界值來進行測試，這樣可能會忽略一些特殊值的測試，例如在日曆相關的程式中 2/28 可能會被忽略。邊界測試較適用於實體數值的測試，例如成績、溫度、速度等，但對於電話號碼、學號等邏輯性的數值則比較不適合。

通常邊界測試與等價分割一起使用，接下來看等價分割。

[!NOTE] 🏠 如[游泳池收費系統](#)，請改以非獨立測試進行。

- [游泳池測試案例](#)
- [三角形測試案例](#)

🧠 概念核對問答 (CCQ 2)

- 單選題：假設某個受測方法接受 3 個彼此獨立的輸入參數。若採用「獨立型強固邊界測試 (Independent Robust BVA)」，其設計出的測試案例數量應為多少？ A) 13
B) 19
C) 125
D) 343

► 點擊查看【概念核對問答】答案與解析

5.3 等價分割測試

等價分割測試法 (equivalence partition testing) 和邊界測試很像，都需要把問題空間做分割。分割後為同一個群的測試資料在測試上是等價的，僅需找一個作為代表測試即可。採用等價分割有幾個原因與動機：

- 希望能做完整的測試（而不是僅測試某一偏面）；
- 避免重複的測試。

測試案例是等價的如果

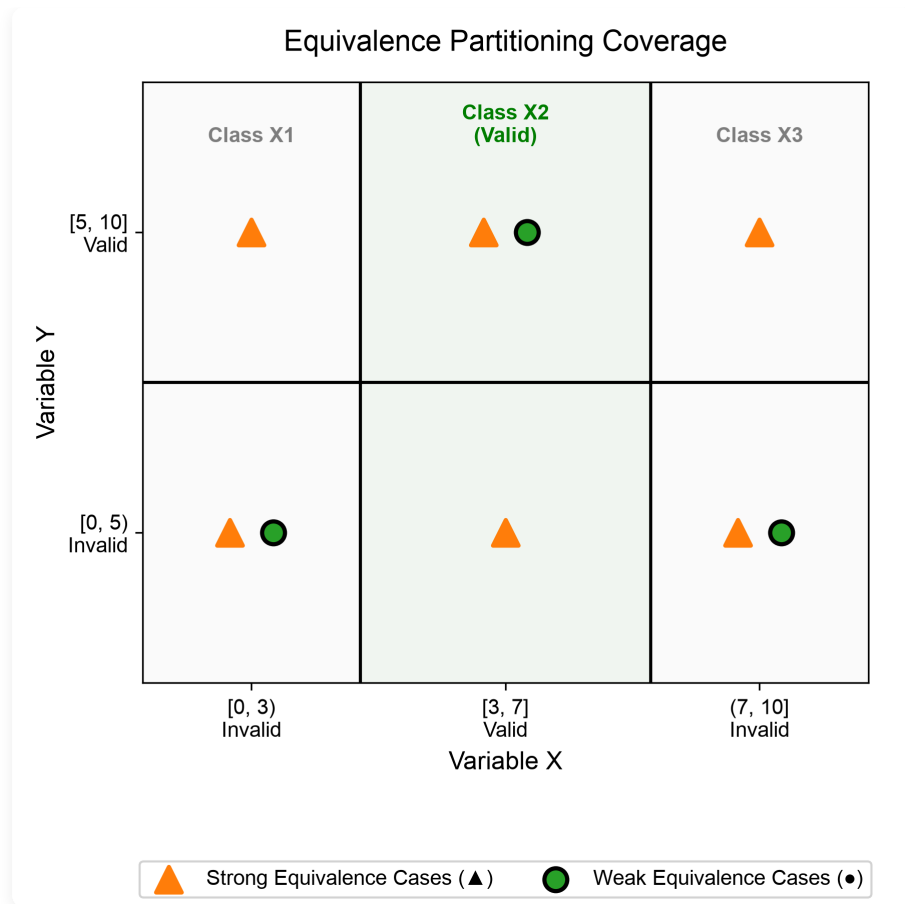
1. 他們是相同的事情;
2. 如果一個可以找到錯誤，其他的也可以;
3. 如果其中一個無法找到錯誤，其他的應該也找不到錯誤。

[!NOTE] **Equivalence partitioning:** A software testing technique that divides the input data of a software unit into *partitions* of equivalent data from which test cases can be derived. In principle, test cases are designed to cover each partition *at least once*. This technique tries to define test cases that uncover classes of errors, thereby reducing the total number of test cases that must be developed. An advantage of this approach is reduction in the time required for testing a software due to lesser number of test cases.

和邊界測試相同，我們需要考慮以下因素。

- 是否考慮強固性：是否考慮每一個變數的例外。
- 輸入、輸出、或是混和：以輸入的變數作為分割的基準，還是以輸出的變數？還是兩者都考慮？分割的基準並沒有一定的標準，需要經驗，但通常可以從輸出的可能性推敲各種需要的輸入組合。
- 獨立或非獨立：如果是獨立變數，我們的測試案例只要能夠「涵蓋」該分割即可（又稱為弱涵蓋 (weak coverage)），其概念相似於邊界測試的獨立測試。若是非獨立，測試案例必須包含所有分割的「乘積」，又稱為 強涵蓋 (strong coverage)。

FIG: 等價分割測試（圓形表示弱涵蓋測試、三角形表示強涵蓋測試）



? 總成績 (grade) 是由考試成績 (exam) 與作業成績 (hw) 合算出來的，其中考試成績佔60分，作業成績佔40分，輸入為 exam 與 hw 兩個成績，輸出為總成績 A, B, C, D 四個等級分別代表 90, 80, 70以上及 70 以下。請利用等價劃分方法建立測試案例。

簡易等價分割

簡易等價分割依據系統規格所制定或描述的「範圍」，簡易的將一個變數分割成數個正常與數個異常的分割，而不對針對工程師可能犯的錯誤或程式架構進行猜測進而設計測試案例。例如上述的例子，exam可以有三分割： e^- , e , e^+ 分表代表 <0 , $0-60$, >60 。hw 可以有三分割： h^- , h , h^+ 分別代表 <0 , $0-40$, >40 等。但這樣單純以輸入變數來分割很容易可以想像是不夠的：因為我們要檢驗分數是否能夠正確的落在 A, B, C, D 區塊中。所以把輸出變數分割也考慮進來： g^- , g^6 , g^7 , g^8 , g^9 , g^+ 分為 <0 , $0-69$, $70-79$, $80-89$, $90-100$, >100 等。

弱涵蓋測試

每一個變數的分割，至少有一個測試案例測試過即可，例如圖 \ref{fig:partition} 中的圓形點，變數 X 有三個分割，Y 有兩個分割，我們只需要 3 個測試案例就可以達到弱涵蓋。亦即，其測試案例數等於變數分割數中最大的值。

在上述成績的例子中，e, h, g 分別有 4, 4, 6 個分割，我們至少需要六個案例來涵蓋所有的狀況。如表為此範例的測試案例。最後的兩個測試案例是考慮一些非法的輸入，例如非數字的例外。

強涵蓋測試

變數分割的乘積（相配），每一個都需要至少有一個測試案例經過，例如圖中的三角形。亦即，測試案例數為變數分割數的乘積。

在上述成績的例子中，如果考慮非獨立（強涵蓋）的情況，則可多達 $3 \times 3 \times 6 = 54$ 測試案例（事實上不會，因為最後一個變數是 output, 而不是所有的 input 組合都能達到各種 output。）。如果增加 a!, b! 表示錯誤的資料型態輸入（字串、亂碼等輸入），則測試案例可能多達 $4 \times 6 = 24$ 個狀況。相較於弱涵蓋的八個測試案例要多出很多。

FIG: 成績程式的等價分割

x,y	e ⁻	e	e ⁺	e!	h ⁻	h	h ⁺	h!	g ⁻	g ⁶	g ⁷	g ⁸	g ⁹	g ⁺	Out
-5,-5	v				v				v						!
30,35		v				v				v					A
30,45		v				v					v				B
30,55		v				v						v			C
40,52		v				v							v		D
50,70			v				v							v	!
y,70				v											!
30,p								v							!

Binary-Search

[!NOTE] 🍊 Binary Search

針對以下 Binary Search 的設計規格（注意這不是程式碼，裡面甚至也沒有演算法），依據等價分割的方法來進行測試案例設計。

```
procedure Search (Key: int ; A: int_array) --> (Found : boolean; L: int)
Post-condition
* 元素被找到，L是索引 ( Found and A (L) = Key ) ，或
* 元素不在陣列中 ( not Found and not
(exists i, 0 <= i <= len(A)-1, A (i) = Key ) )
```

首先我們先思考這個系統會有哪些的輸入輸出？輸入有 key 與陣列。key 本身沒有什麼好分割的，因此考慮陣列，依據題目的前置條件規定，陣列至少有一個元素，所以「一個元素」與「多個元素」應該屬於不同的分割。再考慮輸出來分割，可能有找到或沒找到，因此是兩個分割。在有找到的情

況下位置又是另一個分割，可能是在陣列的第一個、最後一個或是中間的位置，會這樣分割是依據經驗判斷頭尾的位置可能會被忽略（邊界測試）。

總結來說，我們將陣列元素分為一個元素（ a^1 ）、多個元素（ a^* ）與 0 個元素（ a^0 ）。結果可能有找到（ f^1 ）或沒有找到（ f^0 ），輸出的 key 元素可能是在陣列的第一個位置（ c^1 ）、中間位置（ c^m ）或是最後的位置（ c^l ）。 $k!$ 表示輸入的 key 值不符合要求的型態或範圍， $!$ 表示系統會做例外處理。等價分割如下：

- R_1 : 陣列只有一個元素，有找到
- R_2 : 陣列只有一個元素，沒找到
- R_3 : 陣列有多個元素，有找到，在第一個位置
- R_4 : 陣列有多個元素，有找到，在中間位置
- R_5 : 陣列有多個元素，有找到，在最後位置
- R_6 : 陣列有多個元素，沒找到
- R_7 : 陣列沒有元素，沒找到
- R_8 : 不正確的值

下表為依據此分割所做設計，可以注意到每個我們所設定的變數分割都有被涵蓋到，是屬於「弱涵蓋」。我們可能把陣列多個元素在分割為「奇數」與「偶數」，因為程式可能是用 binary search 的方式進行，奇數與偶數可能有所差異。

FIG: Binary Search 的等價分割測試（ k : key; a : array; f : found; c : 位置）

	K	Array	k	k!	a^1	a^*	a^0	f^1	f^0	c^1	c^m	c^l	!
R_1	7	7	v		v			v		v			
R_2	7	8	v		v				v				
R_3	7	7,9,11	v			v		v		v			
R_4	9	7,9,11	v			v		v			v		
R_5	11	7,9,11	v			v		v				v	
R_6	8	7,9,11	v			v			v				
R_7	7		v				v		v				
R_8	a	7,9,11		v		v							v

上例並沒有考慮陣列有偶數個數的情況，對於 binary search 而言，偶數奇數的差別性是有的。另外，也沒有考慮到 key 是位於奇數位置或是偶數位置。請修改此測試案例，包含此兩種分割。

考慮一個三角形判斷的程式，輸入三邊長 a, b, c 輸出為正三角形、等腰三角形、不等邊三角形等三種可能。請依據等價分割設計你的測試案例。

如果我們只從 output 來做測試，設計的測試案例雖然能夠涵蓋四種可能，但還是不足的，例如你可能設計 $a + b < c$ 的案例，測試的結果是滿足你的期待：非三角形，但因為沒有從 input 的角度來思考，忽略了 $b + c < a$ 的可能。

Tomorrow

[!NOTE] 🏀 nextDate()

考慮 1812- 2012 年的任何日期，輸入一個日期，輸出他的下一天日期。注意：四的倍數是潤年，但 1900 除外。

簡易分割

先考慮變數，在這個系統中有哪些變數？每個變數的範圍為何？Month: 1-12, Year: 1812-2012, Date: 1-31：

- $M_1 = \{\text{month} \mid 1 \leq \text{month} \leq 12\};$
- $M^- = \{\text{month} \mid \text{month} < 1\};$
- $M^+ = \{\text{month} \mid \text{month} > 12\}$
- $D_1 = \{\text{day} \mid 1 \leq \text{day} \leq 31\};$
- $D^- = \{\text{day} \mid \text{day} < 1\};$
- $D^+ = \{\text{day} \mid \text{day} > 31\}$
- $Y_1 = \{\text{year} \mid 1812 \leq \text{year} \leq 2012\};$
- $Y^- = \{\text{year} \mid \text{year} < 1812\};$
- $Y^+ = \{\text{year} \mid \text{year} > 2012\}$

如果我們採取弱涵蓋只需要 3 個分割測試資料。如果採取強涵蓋則需要 $3 \times 3 \times 3 = 27$ 個測試案例。

很明顯的，光考慮變數的範圍是不夠的，因為這個系統變數間是有關係的（非獨立），例如潤年二月有 29 天。所以我們需要不同的分割。簡易分割雖然可以快速的建立等價分割，但其測試的效益並不高，通常我們需要依據該領域的規則與設計者的經驗進行分割。

- $M_1 = \{\text{month} \mid \text{month has 30 days}\}$
- $M_2 = \{\text{month} \mid \text{month has 31 days}\}$
- $M_3 = \{\text{month} \mid \text{month is February}\}$
- $D_1 = \{\text{day} \mid 1 \leq \text{day} \leq 28\}$
- $D_2 = \{\text{day} \mid \text{day} = 29\}$
- $D_3 = \{\text{day} \mid \text{day} = 30\}$
- $D_4 = \{\text{day} \mid \text{day} = 31\}$
- $Y_1 = \{\text{year} \mid \text{year} = 1900\}$

- $Y_2 = \{year \mid 1812 \leq year \leq 2012 \text{ AND } year \neq 1900 \text{ AND } (0 = year \bmod 4)\}$
- $Y_3 = \{year \mid 1812 \leq year \leq 2012 \text{ AND } 0 \neq year \bmod 4\}$

之所以把 day 分成四個分割，是因為我們知道這四個分割在不同的月份、年份的下一天是有所不同的。例如 2012/2/28 和 2011/2/28 的下一天是不同的、3/30 和 4/30 的下一天是不同的。如果我們採取強涵蓋，應該會有 $343=36$ 個測試案例。

==> [Next Day Example \(slide\)](#)

Swimming-pool-partition-testing

[!NOTE] 🏊 針對[游泳池收費系統](#)，請改以等價分割弱涵蓋方式進行測試

1. 每一個列舉的資料都應該成為一個分割
2. 考慮邊界，將邊界也列為一個分割
3. 請先以 excel 規劃，再以 JUnit 程式碼實踐

更多的等價分割

並不是數字方面的應用才能套用等價分割，等價分可也可以用在列舉條列（各種顏色、各種學制）等情況。另外除了功能測試以外，等價分割也可以應用在系統測試上，例如系統運作在各種不同的瀏覽器、作業系統等。

事實上，等價分割是為種觀念，在每個測試活動中都可應用。

🧠 概念核對問答 (CCQ 3)

- **單選題：**在等價分割測試 (Equivalence Partitioning) 中，「弱 (Weak)」與「強 (Strong)」分類法的主要區別是什麼？
 - A) 「弱」只涵蓋有效等價類，而「強」同時涵蓋有效與無效等價類
 - B) 「弱」基於單一錯誤假設（每個測試案例只測試一個區間的代表值），而「強」則基於多重錯誤假設（測試參數間代表值的笛卡爾積組合）
 - C) 「弱」不需要程式碼規格書，而「強」必須完全依照 SRS
 - D) 「弱」的測試案例數量一定比「強」多

► [點擊查看【概念核對問答】答案與解析](#)

課堂互動

5.4 全成對組合測試

看過前述幾個例子，當我們要執行非獨立測試時，測試案例的成長是很可怕的，例如有 n 個變數，每一個可能有 m 個分割，如果我們每個分割取一個值來做非獨立測試，則有 m^n 個測試案例需要測試。這樣近乎窮盡式的測試方法 在成本可能太高。

[!NOTE] 🖱️ ==All-pairs testing==: A combinatorial method of software testing that, for each pair of input parameters to a system (typically, a software algorithm), tests all possible discrete combinations of those parameters. Also called *pairwise testing*. 成對錯誤假設：錯誤會發生在兩兩配對的情況。

Swimming-Pool-Simple

[!NOTE] 🏊 游泳池收費

凱旋游泳池標準收費為 100 元。(1) 星期六日200元, 除會員以外不打折。(2) 清晨6:00 以前所有人打八折; 中午11:00-13:00 九折。(3) 會員一律打五折。各打折不可重複，以最低價為主。

- d: {d1, d2} 分別代表 星期六日、一般時間。
- t: {t1, t2, t3} 分別代表清晨、中午、其他時段。
- m: {m1, m2} 分別代表會員、非會員。如果採用乘積的方式進行測試，共有 $2*3*2=12$ 個測試案例。下表則是使用全成對的方式。注意日期與時間有乘積、時間與會員有乘積、日期與會員有乘積，故稱為全成對的測試方法，數量也將為六個。當變數變多，可能的狀況變多時，全成對的方法會比全乘積的方式數量少很多。

FIG: 收費系統- 使用全成對方法

	時間	星期	會員
1	t1	d1	m1
2	t1	d2	m2
3	t2	d1	m1
4	t2	d2	m2
5	t3	d1	m2
6	t3	d2	m1

交錯法

把具備最多可能的變數放左邊 (W)，依序把所有變數往右排放。接著放置W 的可能值，w1, w2, w3, 因為他的下一個 X 的可能值是 2 個，所以我們重複的寫上 2個 w1 (行 1, 2)。中間的空行是作為預留之用，往後才會用到。

接著把X 的可能放上去，這時候交錯的放 x1, x2, x1, x2 ... 使得 W 與 X 可以交錯的對應到。

接著放 Y。這時候要注意如果 Y 的還是 y1, y2 依序放的話，x1 永遠都只會對到 y1, x2 只會對到 y2，所以我們在 行4 時做一個對調，這樣 W, X, Y 兩兩成對。接著放 Z。和上面的情況類似，這時候我們 行7,行8 做對調，這樣才會全成對。

	W	X	Y	Z
1	w1			
2	w1			
3				
4	w2			
5	w2			
6				
7	w3			
8	w3			

FIG:

	W	X	Y	Z
1	w1	x1	y1	z1
2	w1	x2	y2	z2
3				
4	w2	x1	y2	z1
5	w2	x2	y1	z2
6				
7	w3	x1	y1	z2
8	w3	x2	y2	z1

假設還有 PQ 兩個變數，也都是有兩個可能的值。這時候會在行1,2 的時候最對調，以完成全成對。

Table: 表格說明

	W	X	Y	Z	P	Q
1	w1	x1	y1	z1	p2	
2	w1	x2	y2	z2	p1	
3						
4	w2	x1	y2	z1	p1	
5	w2	x2	y1	z2	p2	
6						
7	w3	x1	y1	z2	p1	
8	w3	x2	y2	z1	p2	

	W	X	Y	Z	P	Q
1	w1	x1	y1	z1	p2	q1
2	w1	x2	y2	z2	p1	q2
3		x2				q1
4	w2	x1	y2	z1	p1	q1
5	w2	x2	y1	z2	p2	q2
6		x1				q2
7	w3	x1	y1	z2	p1	q1
8	w3	x2	y2	z1	p2	q2

Loan-testing

[!NOTE] 🏀 貸款

是一個放款利率計算的部份介面。已知利率與貸款的金額、償還年限、是否為會員、是否是青年貸款有關係。請應用全成對測試策略，並用交錯法來設計測試案例。

- 假設年數分為兩個分割，金額分為兩個分割，則需要多少個測試案例？測試案例為何？
- 假設年數分為六個分割，金額分為三個分割，則需要多少個測試案例？測試案例為何？
- 若用強涵蓋、弱涵蓋，則需要多少個測試案例？測試案例為何？

FIG: 放款利率計算介面

The image shows a web-based mortgage interest rate calculator. At the top, there is a navigation bar with links for '首頁' (Home), '房貸資訊' (Mortgage Information), and '聯絡我們' (Contact Us). The main heading is '房貸利率計算器'. The calculator form includes the following fields and controls:

- 貸款金額** (Loan Amount): A text input field containing '10,000,000' and a currency selector set to '元' (Yuan).
- 償還年限** (Repayment Term): A text input field containing '20' and a unit selector set to '年' (Years).
- 是否為會員** (Are you a member?): A toggle switch currently set to '是' (Yes).
- 是否為青年貸款** (Are you a young person loan?): A toggle switch currently set to '是' (Yes).
- 計算利率** (Calculate Interest Rate): A large blue button.
- 放款利率: 1.85%** (Interest Rate: 1.85%): The result displayed in large blue text.
- *本利率僅供參考，實際利率依銀行審核為準 (This interest rate is for reference only; the actual interest rate is subject to bank review).

開發與工具

All pair testing 工具的開發：

1. **程式技術**：主要需要組合數學 (Combinatorics)、圖論 (Graph Theory) 的知識，以及能夠高效處理資料結構和實現優化演算法的程式語言（如 Java, Python, C++）。
2. **複雜度**：設計出能生成最小測試案例集的 All-Pairs 測試問題，是 NP-hard 問題。然而，實際使用的生成演算法（如 ACTS 中的 IPO 演算法）是高效的貪婪啟發式 (Greedy Heuristic) 演算法，它們通常能在合理的時間內找到近似最佳解。

工具：

- **PICT** (Pairwise Independent Combinatorial Testing), 容易取得，免費, Windows (命令行工具), 無原生的 Java 版本, 微軟的開源工具，非常流行，但需要 Windows 環境。
- **Allpairs Generator**, 容易取得，免費, Perl 腳本, 無原生的 Java 版本, Perl 寫成的腳本，需要在系統中安裝 Perl 環境。
- **ACTS** (NIST), 容易取得，免費, Java (JAR 檔案), 由美國國家標準與技術研究院 (NIST) 開發的，是 Java 寫成的, 功能強大，支援更高的強度 (t-way)，且為 Java 使用者設計。

🧑 概念核對問答 (CCQ 4)

- **單選題**：全成對測試 (Pairwise / All-Pairs Testing) 能夠大幅縮減測試案例數量，其在工程上的核心理論依據是什麼？
A) 軟體系統中的缺陷通常需要至少三個以上的參數交互作用才會觸發
B) 絕大多數的軟體缺陷都是由「單一變數」或「任意兩個變數之間的交互作用 (2-way Interaction)」所引起的

- C) 配對測試是白箱測試的一種，可以直接涵蓋所有的程式碼分支路徑
- D) 成對測試可以保證 100% 涵蓋多變數系統的所有笛卡爾積組合

► 點擊查看【概念核對問答】答案與解析

課堂互動

5.5 測試案例的優化

如果有五個因子，分別有 4,5,6,7,8 個類別，那們強涵蓋就需要 $4 \times 5 \times 6 \times 7 \times 8 = 6720$ 個測試案例。太過於複雜。

1. 分離例外

- **基本原理：** 嚴格區分**有效等價類**和**無效等價類**，並在無效類中進一步識別出**例外類別**。
- **具體做法：**
 - **扣除等效的例外類別：** 如果所有因子都有**相同的異常行為**（例如，所有因子的輸入超限都會導致相同的錯誤碼），則您不需要為每個因子測試這個異常的組合。您可以將這些異常類別視為一個**單獨的集合**，只在特定的、最能代表異常的組合中測試，或者只測試它們與有效類別的組合。
 - **最小有效集：** 對於 N 個類別，如果 $N - k$ 個類別具有相同的成功/有效行為，則只需要選擇其中一個類別作為**代表**進行有效組合測試。測試數量變成 $(4-2) * (5-2) \dots (8-2) = 720$ 大量的減少測試案例。

2. 配對組合

可以改用 2-way 或是 n-way 的部分組合方式來進行測試。

3. 邊界值分析

結合等價分割後，我們優先測試**邊界值**。

- 缺陷最容易出現在等價類的邊界上。
- 對於每個因子的每個等價類，測試其**精確邊界**、**邊界內一個值**、**邊界外一個值**（例如： N , $N - 1$, $N + 1$ ）。雖然邊界值分析不會直接減少因子數量，但它**取代了**在等價類內部隨機選擇值的需求，讓選取的測試點更有針對性，從而避免了對整個等價類的冗餘測試。

4. 故障模式和影響分析

Failure Mode and Effects Analysis, FMEA

- **基本原理：** 根據每個因子可能導致的風險和影響來分配測試資源。
- **具體做法：**
 - **高風險優先：** 識別出那些**最可能出錯**或一旦出錯會造成**最嚴重影響**的因子和類別。

- **低風險簡化**：對於那些幾乎不可能出錯、或者其錯誤影響很小的因子，可以將其測試降級為僅進行有效類別的組合測試，甚至只進行單個有效值的測試。

概念核對問答 (CCQ 5)

- **單選題**：關於「正交表測試 (Orthogonal Array Testing)」與一般「成對測試 (Pairwise Testing)」的比較，下列敘述何者正確？
A) 正交表測試是隨機產生的，而 Pairwise 必須透過數學嚴格推導
B) 兩者都關注參數間的配對，但正交表更強調各因子組合的「均勻平衡性（正交性）」，而 Pairwise 僅要求任意兩因子的組合至少出現一次，因此 Pairwise 的案例數量通常更少且更有彈性
C) 只要變數的個數相同，正交表與 Pairwise 產生的測試案例清單必定完全一致
D) 正交表測試只能處理二分值（True/False）的變數組合

► [點擊查看【概念核對問答】答案與解析](#)

課堂互動

5.6 決策表測試

如果應用程式有複雜的邏輯規則，變數之間的相依性很強，亦即每個變數是不獨立的，我們可以透過決策表來協助分析這些邏輯。複雜的邏輯規則是很惱人的，決策表的 condition-action rule 可以一目了然的讓我們了解這些規則。

CAR 決策表設計法，其中 ==C: Condition== 表示每一個細項的狀況，==A: Action== 事件表示系統需要進行的動作，==R: Rule== 規則表示在各種不同的狀況下系統應該進行的動作。在 Table 中，每一個行表示一條的規則，例如「當 C1 且 C2 且 C3 時，做 A1」「當 C1 且 C2 但非 C3 時做 A2」等等。

事實上，決策表不僅可用於測試，也適合在「需求分析」階段表達使用者的需求或邏輯。

FIG: 狀況，事件，規則表

條件與事件	規則 1 (R1)	規則 2 (R2)	規則 3 (R3)	規則 4 (R4)
條件 (Conditions)				
C1: 條件 1	Y	Y	N	N
C2: 條件 2	Y	N	Y	N
C3: 條件 3	Y	-	-	Y
動作 (Actions)				
A1: 執行動作 1	X	X		
A2: 執行動作 2		X	X	
A3: 執行動作 3				X

在這個例子中，每一個 condition 都是一個條件為「真假」的變數，其實也可以試一個變數，而條件值為一個數字、範圍或描述性的條件。

在下面三角形的例子中，我們用 C_1 表示 a, b, c 不為三角形的狀況，這樣 CAR 才不會膨脹的太大，而且可閱讀性也高，如果我們把 C_1 拆解為 $C_{11}: a + b \geq c$, $C_{12}: b + c \geq a$, $C_{13}: a + c \geq b$ ，當然也可以，但表會變成很大難以閱讀與分析。

FIG: 三角形的狀況事件規則表 (CAR table)

條件與動作	R1	R2	R3	R4	R5	R6
條件 (Conditions)						
C1: $a < b + c$ 且 $b < a + c$ 且 $c < a + b$?	N	Y	Y	Y	Y	Y
C2: $a = b$?	-	Y	Y	N	N	N
C3: $b = c$?	-	Y	N	Y	N	N
C4: $a = c$?	-	Y	N	N	Y	N
動作 (Actions)						
A1: 輸出「非三角形」	X					
A2: 輸出「等邊三角形」		X				
A3: 輸出「等腰三角形」			X	X	X	
A4: 輸出「不等邊三角形」						X

在 nextDay() 的例子中，輸入一個日期，輸出是以下幾個動作是否執行：*impossible, increment day, increment month, increment year, reset day, reset month}。請以決策表設計測試案例。

FIG: nextDay() 等價類劃分

變數 (Variable)	有效等價類 (Valid Equivalence Classes)
月份 (Month)	M1 = {30天月份: 4, 6, 9, 11} M2 = {31天月份: 1, 3, 5, 7, 8, 10, 12} M3 = {二月: 2}
日期 (Day)	D1 = {1 ≤ day ≤ 28} D2 = {day = 29} D3 = {day = 30} D4 = {day = 31}
年份 (Year)	Y1 = {世紀平年: year = 1900} Y2 = {一般閏年: 1812 ≤ year ≤ 2012 且 year ≠ 1900 且 year mod 4 = 0} Y3 = {一般平年: 1812 ≤ year ≤ 2012 且 year mod 4 ≠ 0}

- M1 = {month: month has 30 days}
- M2 = {month: month has 31 days}
- M3 = {month: month is February}
- D1 = {day: 1≤day≤28}
- D2 = {day: day=29}
- D3 = {day: day=30}
- D4 = {day: day=31}
- Y1 = {year: year = 1900}
- Y2 = {year: 1812≤year≤2012 AND (year!=1900) AND(year=0 mod 4)}
- Y3 = {year: (1812≤year≤2012 AND year!=0 mod 4)}

FIG: 應用決策表測試法於 nextDay()

規則 (Rules)	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10
條件 (Conditions)										
月份 (Month)	M1	M1	M1	M1	M2	M2	M2	M2 (Dec)	M2 (not Dec)	M2 (not Dec)
日期 (Day)	D1/D2	D3	D4	-	D1/D2/D3	D4	-	D4	D4	D4
年份 (Year)	-	-	-	-	-	-	-	-	-	-
動作 (Actions)										
A1: 日期加 1	X				X					
A2: 月份加 1, 日期歸 1		X							X	
A3: 年份加 1, 月/日歸 1								X		
A4: 錯誤/不可能的日期			X							

❓ 透過上述的決策表我們發現有些地方是無法確定的（打X的部份），因此分割需要再細一些，怎麼做呢？有需要修改分割嗎？

🧠 概念核對問答 (CCQ 6)

- **單選題：**在軟體測試實務中，下列哪一種受測情境最適合優先採用「決策表測試 (Decision Table Testing)」來設計案例？
 - A) 系統輸入參數彼此完全獨立，且有連續性數值邊界
 - B) 輸入參數之間存在複雜的商務邏輯與制約關係，不同的條件組合會觸發不同的系統動作或輸出結果
 - C) 系統的輸出僅與目前輸入值有關，與輸入條件的組合邏輯無涉
 - D) 系統的運作強烈依賴時間序列與物件歷史狀態的轉移

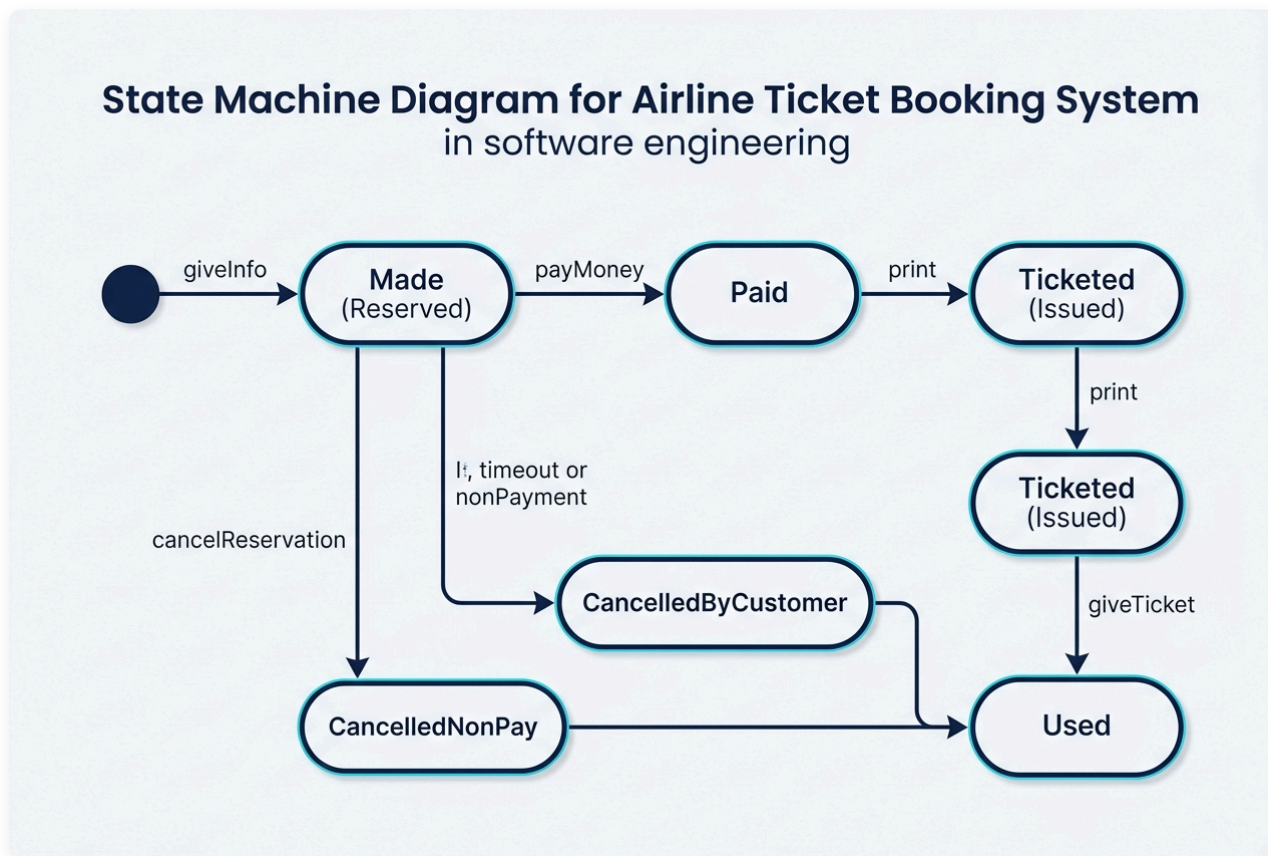
► 點擊查看【概念核對問答】答案與解析

課堂互動

5.7 狀態測試

沒有 return 的 function 怎麼測試？

前面我們所提到的測試多半著重在於輸入輸出的檢查，但是現在的軟體系統很多都是事件導向（event driven）的，系統內部會有一個狀態機（state machine）來記錄現有的狀態，並決定接受到狀態時應該做出什麼反應，轉移到哪一個狀態。像這一類的系統測試的重點就不是在功能是否正確，而是狀態的反應是否正確。



圖形解說：航空訂票系統狀態機 (State Machine Lifecycle)

- 主流程 (Happy Path) :
 - i. **Made (已預約)**：接收 `giveInfo` 事件後建立訂位。
 - ii. **Paid (已付款)**：接收 `payMoney` 事件後完成扣款。
 - iii. **Ticketed (已開票)**：接收 `print` 事件後完成開票。
 - iv. **Used (已使用)**：接收 `giveTicket` 登機使用後順利結案。
- 取消與例外分支：
 - **CancelledByCustomer (顧客主動取消)**：在 **Made**、**Paid**、**Ticketed** 狀態下均可觸發 `cancel` 轉移至此狀態。
 - **CancelledNonPay (未付款逾期取消)**：在 **Made** 狀態下因付款超時 (`payTimeExpire`) 自動轉移至此狀態。

圖表示一個飛機票的訂票系統，我們先把他的狀態定義出來：`made` (預約), `paid` (已付款), `ticketed` (已開票), `used` (已使用), `cancelled by customer` (顧客取消), `cancelled nonpay`

(未付款取消)。狀態轉移 (transition) 上的格式為：

表示狀態接收到事件E 後若滿足條件 C 則會進行動作 A，並且進行狀態轉移。

測試時有以下涵蓋度的作法：

- 狀態涵蓋度 (state coverage)：每個狀態都被涵蓋。
- 事件涵蓋度 (event coverage)：每個事件都被涵蓋。
- 狀態轉移涵蓋度 (state transition coverage)：每個狀態轉移都被涵蓋。

FIG: 全狀態轉移涵蓋

百分百狀態轉移涵蓋並不保證百分百事件涵蓋，因為可能有多個事件會造成相同的狀態轉移。

為了達到全狀態與全狀態轉移的涵蓋，我們設計了以下 5 條測試路徑（事件序列）：

- 路徑 1：giveInfo -> payMoney -> print -> giveTicket (涵蓋 Made, Paid, Ticketed, Used)
- 路徑 2：giveInfo -> payMoney -> print -> cancel (涵蓋 Made, Paid, Ticketed, CancelledByCustomer)
- 路徑 3：giveInfo -> payMoney -> cancel (涵蓋 Made, Paid, CancelledByCustomer)
- 路徑 4：giveInfo -> cancel (涵蓋 Made, CancelledByCustomer)
- 路徑 5：giveInfo -> payTimeExpire (涵蓋 Made, CancelledNonPay)

注意其中第五個測試案例需要產生過期未繳款的事件，這可能需要一段不短的時間（例如一個星期），會對測試造成困擾，所以通常需要假時間的建立以加快測試的效率。

設計檢驗

為了避免設計的疏忽造成狀態轉移的缺失，我們可以使用狀態轉移表來檢查 狀態對每一個事件是否都處理了？如下圖表示 s1 接受 e1, e2 訊息會轉移到 s2, 對於 e3, e4 等事件則沒有反應。

狀態轉移表:

	e1	e2	e3	e4
s1	s2	s2	x	x
s2	x	s3	x	x
s3	x	x	s1	x

物件狀態測試

前面 JUnit 的測試都著重在某方法（副程式）的功能測試，但現在的系統多半是物件導向，只測試功能導向式不夠的。回想一下，我們用 JUnit 測試時，都針對有回傳值的功能做檢驗，例如 `assertEquals(12, a.m1())`。如果 `m1()` 的方式並沒有回傳值，難道我們就不用測試了嗎？

在物件狀態測試中，我們要檢驗的是「物件的狀態是否如我們預期的變化」。例如執行完 `m1()` 後物件的狀態變為 `isComplete`，我們就可以透過 `assertTrue(a.isComplete())` 來檢驗。物件內部的屬性代表著它的狀態，「執行動作、改變狀態」，是物件開發的特性之一。

我們來看看如何應用 JUnit 進行物件的狀態測試。

Stack-test

[!NOTE] 🍌 test Stack

設計一個 Stack 物件，裡面用陣列來儲存內容。有以下的介面：

- `Stack(int n)`：宣告裡面內容為 `n` 個的 Stack;
- `int pop() throws StackEmptyException()`：當系統已經在 `empty` 的狀態又 `pop` 的話就會拋出例外;
- `void push(int x) throws StackFullException()`：當系統已經在 `full` 的狀態又 `push` 的話就會拋出例外;
- `boolean isEmpty()`：檢驗是否為空 Stack;
- `boolean isFull()`：檢驗是否塞滿元素。

1. 畫出 Stack 的狀態圖
2. 針對這個 Stack 設計測試案例，使得全狀態轉移涵蓋。
3. 以 JUnit 來進行此測試
4. 請檢驗程式碼之涵蓋度

在多數的情況，我們必須先做一些基本的資料設定（或是初始狀態的設定），才開始進行事件的輸入。例如要測試一個 Stack 的反應，我們要先設定 Stack 內有多少元素，例如是 3 個，那們我們給予連續三個 `pop` 的事件後，該物件應該來到 `Empty` 的狀態。

設計測試案例達到 `isFull` 的狀態:

```
Stack s = new Stack(3);
s.push(100);
assertFalse(s.isFull());
s.push(200);
s.push(300);
assertTrue(s.isFull());
```

設計測試案例達到 `isEmpty` 的狀態

```
Stack s = new Stack(3);
s.push(100);
s.push(100);
s.pop();
s.pop();
assertTrue(s.isEmpty());
```

😓 我是一個苦命的程式設計師，今晚加班到快通宵了，睏得快睜不開眼了，女上司很關心，問我要不要吃宵夜。我沒好氣地說，宵夜就算了，能讓我睡一覺就行了。女上司紅著臉說了句討厭啊，然後坐在我身邊不動，距離我很近，搞得我很緊張... 難道她發現我的程式有 bug？

See [DemoJUnit](#) 下 `state/Stack` 的完整程式碼與測試碼。

🧠 概念核對問答 (CCQ 7)

- **單選題：**在狀態測試 (State Testing) 中，關於「狀態覆蓋 (State Coverage)」與「轉移覆蓋 (Transition Coverage)」的強度關係，下列敘述何者正確？
A) 達到狀態覆蓋必定代表同時達到了轉移覆蓋
B) 轉移覆蓋的強度大於狀態覆蓋；若測試案例達到了轉移覆蓋（驗證了所有可能的轉移路徑），則必定已涵蓋了所有狀態（達到狀態覆蓋）
C) 兩者互相獨立，沒有任何包含或強弱關係
D) 狀態測試不需要考慮無效轉移（即在某狀態下輸入非法事件的反應）

► 點擊查看【概念核對問答】答案與解析

課堂互動

5.8 屬性基礎測試 (Property-Based Testing)

在傳統的單元測試中，我們習慣採用 **範例基礎測試 (Example-Based Testing, EBT)**。我們手動挑選幾組具代表性的輸入資料（如邊界值、等價區間內的數值），然後寫出對應的預期輸出，並使用 `assertEquals` 等斷言進行驗證。然而，這種做法高度依賴測試工程師的經驗，容易漏掉那些難以預料的極端輸入 (Edge Cases)。

屬性基礎測試 (Property-Based Testing, PBT) 則是另一種典範轉移：它要求我們不去定義「具體的測試資料與預期輸出」，而是定義**「程式碼應該永遠滿足的數學屬性或狀態不變量 (Class/Algorithmic Invariants)」**。接著由測試框架自動生成成千上萬組不同的極端隨機測資，來試圖找出讓這個屬性失敗的反例。

💡 屬性基礎測試先驅：John Hughes 與 QuickCheck

屬性基礎測試 (PBT) 最初由瑞典查爾摩斯工學院教授、ACM Fellow John Hughes 與 Koen Claessen 於 2000 年共同提出，並發表了劃時代的函式庫 QuickCheck (榮獲 ACM SIGPLAN 最具影響力論文獎)。

John Hughes 的核心倡議是：「不要手寫測試案例，定義程式屬性！(Don't write tests, specify properties!)」

他認為傳統範例測試讓工程師浪費過多時間在思考個別測資，卻依然無法防範極端情境；PBT 讓人類回歸高層次的抽象規格思考，並交由電腦自動生成萬組隨機輸入。更關鍵的是，Hughes 發明了**「測資收縮 (Shrinking)」** 演算法，當隨機測資中觸發錯誤時，框架會自動二分逼近並縮減成最小反例（例如將長度 100 的亂碼陣列縮減至 `[0, -1]`），讓工程師能在數秒內抓出缺陷根本原因。現代主流生態系中的 Java `jqwik`、Python `Hypothesis` 與 TypeScript `fast-check`，皆源於 QuickCheck 的設計哲學。

5.8.1 核心概念：不變量與隨機測資

1. 屬性 (Property) / 不變量 (Invariant)：

- 指的是不論輸入什麼資料，程式執行完畢後都必須成立的規則。
- 例如排序演算法：給定任何 List `A`，排序後的 List `B` 必須滿足：
 - `B` 的長度必須與 `A` 相同。
 - `B` 內的所有元素必須與 `A` 相同（是 `A` 的排列組合）。
 - 對於 `B` 中任何相鄰元素 `B[i]` 與 `B[i+1]`，都必須滿足 $B[i] \leq B[i+1]$ 。

2. 自動生成測資 (Generator)：

- 框架（如 Java 的 `jqwik`、Python 的 `Hypothesis`）會自動生成隨機字串、極大/極小整數、包含空值或重複元素的陣列等。

3. 收斂 / 縮小化 (Shrinking)：

- 當隨機生成的一萬組資料中，有一組極為複雜的資料導致測試失敗時，AI 框架會自動對該資料進行「簡化」，嘗試以最簡短、最容易讓人類看懂的「極簡反例」回報給開發者。
- 例如：如果長度為 100 且包含特殊字元的隨機陣列在排序時出錯，PBT 框架經過 Shrinking 後，最終可能只會回報 `[0, -1]` 這樣簡單的反例，幫助工程師精準除錯。

5.8.2 `jqwik` 實戰範例 (Java)

以下是一個使用 `jqwik` 框架測試「整數加法交換律」與「相加不變量」的簡單範例：

```
import net.jqwik.api.*;

public class AdditionProperties {

    // 測試加法交換律：a + b 必須等於 b + a
    @Property
    void additionIsCommutative(@ForAll int a, @ForAll int b) {
        int result1 = a + b;
        int result2 = b + a;
        assert result1 == result2;
    }

    // 測試加法恆等律：任何數加上 0 必須等於自己
    @Property
    void additionWithZeroHasNoEffect(@ForAll int a) {
        assert a + 0 == a;
    }
}
```

- **@Property**：告訴 jqwik 這是一個屬性測試，預設會自動執行 1000 次隨機測試。
- **@ForAll**：指示框架自動為該參數生成各類極端值（包括整數最大值 `Integer.MAX_VALUE`、最小值、零、負數等）。

🔧 實習手冊連結：

- Property-Based Testing 實戰：[../../LabDemo/docs/u04_utest/jqwik_property_based.md](https://github.com/qq123456789/LabDemo/docs/u04_utest/jqwik_property_based.md)

🧠 概念核對問答 (CCQ 8)

在屬性基礎測試 (Property-Based Testing) 中，我們不需要手動為每一組測試寫出確切的預期輸出數值，而是定義程式執行時必須永遠維持的「屬性或不變量」，並交由測試框架隨機生成大量測資來尋找反例。

A) 正確 (True) B) 錯誤 (False)

► 點擊查看【概念核對問答】答案與解析

課堂互動

✨ Check

1. T/F: 規格書對於黑箱測試尤為重要，因為測試案例的設計通常是基於規格書的。

2. 黑箱測試與白箱測試的差異為何？

3. 關於邊界測試，以下何者正確？

- 強固邊界測試考慮 $\max+$ 。
- 非獨立邊界測試是在滿足「單一錯誤假設」的條件下進行的測試。
- 非獨立強固邊界測試其測試案例個數最少為 $7^n + 1$ ，其中 n 為變數的個數。
- 獨立型邊界測試其測試案例最少為 $6 \times n + 1$ 。

4. 關於等價分割測試，以下何者為真

- 透過檢視程式碼中的判斷句，來進行測試案例的等價分割;
- 如果分為 n 個等價，我們每一個等價中取出 n 個測試案例來進行測試;
- 不只考慮輸入，輸出也應該列入考慮來設計等價;
- 假設影響等價分割的因子有三個，其個別的分割有 5, 4, 6 個，則弱涵蓋的等價個數約等於 6，強涵蓋的等價個數約 120 個。

5. 以下何者為真？

- 全成對組合測試不但提高測試的有效度，也簡化測試的負擔。
- Decision table testing 適用於複雜的企業規則，特別是輸入是由許多不同的狀況組合而成的情況。
- 等價分割測試是把程式分成若干可測試的模組，藉此降低測試的複雜度。
- 等價分割測試的強涵蓋會比弱涵蓋需要更多的測試案例。

6. 關於全成對組合測試，以下何者為真（選二）：

- 假設錯誤會出現在兩兩變數交錯運算比對上;
- 好處之一是不需要進行等價分割;
- 可視為弱涵蓋與強涵蓋之間的測試案例設計;
- 測試案例的個數約等於最大等價個數與最小等價個數的平均值。

7. 關於狀態測試，以下何者為真？

- 狀態轉移圖 (STD) 可以表達系統的行為，也是擷取需求很好的方式;
- 狀態表可以檢驗狀態圖是否設計周全;
- 狀態涵蓋度百分百，則事件涵蓋度必定百分百。
- 對於 Stack 而言，push 是一個事件，full 是一個狀態。

練習

5.ex.spec 設計規格

一個功能模組的「規格」包含該模組的參數、回傳型態、前置條件、後置條件。寫出以下功能的規格，嘗試以正規的方式來描述之。並設計測試案例。

1. 由一群陣列中找出最大的值回傳
2. 針對一個陣列做由小到大的排序
3. 輸入一段文字和一個單字，回傳該單字在該文字出現的位置
4. 輸入兩個字串型態的實數，回傳其相加的結果

5.ex.pool 游泳池收費

以下是一個游泳池收費系統的規則：(1) 一般票價 200 (2) 星期六日 250 元，除會員以外不打折 (3) 12 歲以下、60 歲（含）以上打八折 (4) 七點以前八折 (5) 團體打七折 (6) 會員打五折 (7) 各打折不得重疊使用。請依據以下方法進行測試，請先估算有多少測試案例，在把測試資料寫出來。

1. 強固邊界測試
2. 非獨立強固邊界測試
3. 請撰寫程式碼與並應用該測試策略設計 JUnit 測試碼，並進行測試。

[!NOTE]

5.ex.loan 青年安心成家

青年安心成家 (112-1 期末考題):

公股銀行辦理青年安心成家購屋優惠貸款，貸款說明如下：貸款標的：申請日前6個月起所購置之住宅。貸款對象：借款人符合民法規定之成年年齡以上，且借款人與其配偶及未成年子女均無自有住宅者。

- 貸款成數：最高 8 成核貸。
- 貸款額度：最高新臺幣1000萬元。貸款年限及償還方式：貸款年限最長40年，含寬限期5年，本息分期平均攤還或本金分期平均攤還。
- 貸款利率：以中華郵政股份有限公司2年期定期儲金額度未達新臺幣500萬元機動利率為基準利率，自112年3月29日起為1.595%，優惠為 1.22%。計息方式採一段式機動利率、二段式機動利率 (TwoPhaseRating) 或混合式固定利率擇一，一經選定不得變更。各利率簡述如下：
 - 一段式機動利率：按基準利率固定加0.585%（目前為1.805%）機動計息。但自110年1月1日起新貸放案件，按基準利率固定加0.555%（目前為1.775%）機動計息。
 - 二段式機動利率：前 2 年按基準利率固定加0.345%（目前為1.565%）機動計息，第3年起按基準利率固定加0.645%（目前為1.865%）機動計息。
 - 混合式固定利率：前2年採固定利率，第1年按「撥貸當時」基準利率固定加0.525%（目前為1.745%）固定計息，第2年按「撥貸當時」基準利率固定加0.625%（目前為1.845%）固定計息，第3年起按基準利率固定加0.645%（目前為1.865%）機動計息。

今欲開發程式協助人民計算每月所需繳費的金額。程式分為兩部分，第一部分為檢驗資格 (`checkQualification()`)，第二部分為計算每月所需要的繳費 (`monthlyPay()`)。在程式開發之前，我們預先規劃採用 等價分割 進行有效的測試案例規劃，請

- 針對檢驗資格(`checkQualification()`)採用強涵蓋 (strong coverage) 的方式進行案例規劃;
- 針對每月所需要的繳費採取弱涵蓋 (weak coverage)
- 針對每月所需要的繳費採取 全成對 (all pair testing) 或強涵蓋擇一的測試案例規劃。

注意：

- 你不需要提交程式碼; 請用 excel 進行測試案例的規劃，不需寫出預期輸出。
- 做有效的測試規劃，並不是題目中所有的變數都必須是測試參數。

參考解答

5.ex.watch 電子表

手錶。b_1 會切換正常顯示與鬧鐘模式。b_2 會閃爍「小時」，這時候如果 b_3 會增加小時，b_4 會減少 1 小時，b_1 或超過 10 秒鐘會回到不閃爍。

- 畫出狀態圖;
- 檢驗你的狀態圖，是否清楚交代每個狀態對於不同事件的處理？
- 寫出測試案例，完成狀態轉移涵蓋;
- 寫出 JUnit 程式碼;
- 寫出程式碼;
- 完成測試。

5.ex.triangle 三角形

寫一個程式 checkTriangle(int, int, int) 來判斷是何種三角形（正三角、等腰三角、一般三角、非三角形），邊長介於 1-200 之間。

- 請以獨立強固邊界測試設計測試案例，並將測試案例以 testcase.txt 來編寫，其格式為如下，表示 100, 100, 100, 會回傳 正三角，1, 2, 100 會回傳 !。每一個測試案例以換行來區分。

```
100, 100, 100, 正三角
1, 2, 100, !
```

- 同上，請以非獨立強固邊界測試設計之。
- 請設計 JUnit 來進行測試，JUnit 會讀取 testcase.txt 來做為測試資料。

5.ex.gem 逢甲體育館

- 逢甲體育館設有室內跑道一圈，今在兩端各設計一個讀卡機，分別為 A, B 讀卡機，學生經過時刷卡作為記錄以培養學生運動風氣。系統會儲存以下的資訊：學生 ID, 時間, 讀卡機 ID。學生獲取紅利的規則如下：
 - 半小時內需要有交錯的讀卡機記錄 4 次以上，例如 ABAB，如此則算為一次運動，獲得一點。
 - 一天內可以有多次運動，但間隔必須為一小時才能算兩次，否則視為一次。一日最多算計 5 次運動。
請先描述你的測試策略或方法（儘量用上課方法），再設計測試案例來測試此系統是否正確。考慮的越詳細越好，必要時做適當的假設。
 - Hint: 可先僅考慮都是同一個學生，所以測試案例不用寫學生 ID。例如 tc1={ (9:00A), (9:10B), (9:20A), (9:30B) } 表示四筆資料的 log。
 - Hint: 測試的重點包含 (1) 資料交錯錯誤時是否能夠判定正確 (2) 超過五次運動的計算是否正確 (3) 間隔的計算是否正確等。
 - Hint: 最容易被忽略的是是否能夠為學生爭取到最高的積點，例如學生運動了五次，第一次與第二次相隔較久導致前面的四次沒有辦法在半小時內完成，但若從第二次算起則可以有

一次的積點。

5.ex.xyz

有一個系統的輸出取決於三個變數 X, Y, Z。X可能的值 x1, x2, x3; Y 可能的值為 y1, y2; Z可能的值為 z1, z2。針對弱涵蓋、全成對、強涵蓋三種等價分割方式

1. 三種方法，分別需要最少設計多少測試案例？
 2. 請用這三種方法規劃測試案例;
-